



Preview js\_syllabus\_phase2.md ×

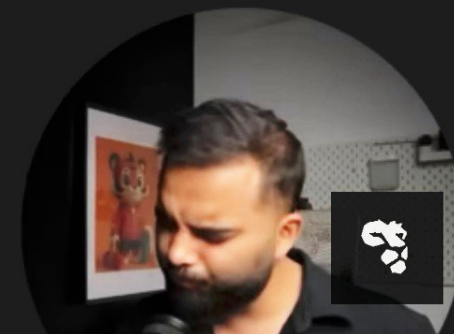
JS script.js

index.html



## 1. The DOM (Document Object Model)

-  Teach:
  - DOM Tree structure: node, element, text, comment
  - Selecting elements: getElementById, getElementsByClassName, querySelector, querySelectorAll
  - Text/content access: innerText, textContent, innerHTML
  - Attribute manipulation: getAttribute, setAttribute, removeAttribute
  - Dynamic DOM manipulation: createElement, appendChild, removeChild, prepend
  - Style updates via .style and classList (add, remove, toggle)
-  Confusion:
  - Difference between innerText vs textContent vs innerHTML
  - Live HTMLCollection vs static NodeList
-  Mindset:
  - DOM is the page's brain – use it responsibly



## 2. Events and Event Handling

1.5x

-  Teach:
  - Event binding: `addEventListener`, `removeEventListener`
  - Common events: `click`, `input`, `change`, `submit`, `mouseover`, `keyup`
  - Event object: `target`, `type`, `preventDefault`
  - Event bubbling and capturing
  - Event delegation
-  Confusion:
  - `event.target` vs `event.currentTarget`
  - Capturing phase vs bubbling phase
-  Mindset:
  - Don't bind events everywhere, delegate smartly
-  Practice:
  - Live character counter
  - Delegated event handler on todo list



## 3. Forms and Form Validation

-  Teach:
  - Reading values from input, textarea, select
  - Prevent default submission
  - Inline and JS-based validation
  - Showing error messages conditionally
  - Pattern attribute vs custom regex
-  Confusion:
  - Difference between value and textContent
  - Form submission vs button click
-  Mindset:
  - Forms are for communication – sanitize and validate deeply
-  Practice:
  - Email/password validator
  - Interactive feedback form with error highlights

## 🕒 4. Timers and Intervals

- ✅ Teach:
  - `setTimeout`, `clearTimeout`
  - `setInterval`, `clearInterval`
  - Real use: delaying UI actions, auto-refresh
- ⚠️ Confusion:
  - `setInterval` vs `setTimeout` recursion
- 🧠 Mindset:
  - Time creates rhythm in user experience
- ✍️ Practice:
  - Countdown timer
  - Auto-hide alert banner after 3s



## 5. LocalStorage, SessionStorage, and Cookies

-  Teach:
  - localStorage API: setItem, getItem, removeItem, clear
  - sessionStorage API
  - Storing/retrieving strings vs JSON
  - Basic cookie structure (manual key=value;path=/ format)
-  Confusion:
  - Why only strings work in localStorage
  - Cookies need manual encoding/expiration handling
-  Mindset:
  - Use localStorage for state, cookies for cross-tab auth
-  Practice:
  - Save theme preference in localStorage
  - Login form that remembers user's name using storage

## Bonus Practice Project Ideas for Phase 2

-  DOM Interactivity:
  - Todo App (add/delete/edit)
  - Accordion / collapsible FAQ
-  Event Handling:
  - Real-time search filter
  - Custom dropdown menu
-  Form Management:
  - Survey form with multiple fields and validation
-  Storage Integration:
  - Notes App (save text to localStorage)
  - Dark mode toggle saved across sessions



Nice — these slides cover a compact Phase-2 JavaScript syllabus. I'll walk through **each topic** from the images and explain the concepts, pitfalls, mental models, and practical code examples you can reuse. I'll also expand the practice project ideas with implementation tips and what to focus on while building them.

# 1. The DOM (Document Object Model)

## What it is / mindset

- The DOM is the browser's representation of the page — think of it as the page's brain or an object tree of everything on the page. Every HTML element becomes a node in this tree. Treat DOM operations as state changes: manipulate nodes to change UI.

## DOM tree structure

- Node types: `Element` (tags), `Text` (text nodes), `Comment`, `Document`.
- Parent / child / sibling relationships matter for traversal and event behavior.

## Selecting elements

- `document.getElementById('id')` — returns a single `Element` or `null`. Fast and simple.
- `document.getElementsByClassName('cls')` — returns a *live* `HTMLCollection` (updates when DOM changes).
- `document.getElementsByTagName('tag')` — also a *live* `HTMLCollection`.
- `document.querySelector(selector)` — returns the *first* match (CSS selector). Returns `null` if none.
- `document.querySelectorAll(selector)` — returns a *static* `NodeList` (doesn't update automatically). Useful for iterating.

## Text/content access

- `element.innerText` — returns “rendered” text, respects CSS like `display:none` and line breaks. Retrieving causes reflow in some browsers (slower).
- `element.textContent` — returns the text content exactly as in the DOM (faster), ignores CSS rendering differences.
- `element.innerHTML` — gets/sets HTML markup inside an element (dangerous if used with untrusted input — risk of XSS). Use to insert HTML snippets.

## Attribute manipulation

- `element.getAttribute('attr')`, `element.setAttribute('attr', value)`, `element.removeAttribute('attr')`.
- For many common attributes there are direct properties too: `input.value`, `img.src`, `element.id`, `element.className`. Use `getAttribute` when you need the exact attribute string or data-attributes: `e1.dataset.foo`.

## Dynamic DOM manipulation

- Create and insert nodes:

```
js

const p = document.createElement('p');
p.textContent = 'Hello';
parent.appendChild(p);      // add at end
parent.insertBefore(p, refEl); // insert before refEl
parent.prepend(p);          // add at beginning
child.remove();             // remove itself
parent.removeChild(child);   // older form
```

- For many updates, `innerHTML += ...` is convenient but re-parses whole contents and can wipe event listeners on children.

## Style updates

- `.style` — inline style: `el.style.backgroundColor = 'red'`.
- `.classList` — preferred for toggling styles: `el.classList.add('active')`, `.remove()`, `.toggle()`. Keeps visual styling in CSS and logic in JS.

## Confusions to avoid

- **innerText vs textContent vs innerHTML** — choose based on whether you need rendered text, raw text, or HTML markup.
- **Live HTMLCollection vs static NodeList** — `getElementsByClassName` and `getElementsByTagName` return *live* collections that reflect DOM changes; `querySelectorAll` returns a snapshot. Use `querySelectorAll` if you want to iterate safely without unexpected changes while looping.

# 2. Events and Event Handling

## Event binding

- `element.addEventListener('click', handler)` — recommended; allows multiple listeners and easy removal (`removeEventListener` with same reference).
- Inline handlers (e.g., `onclick="..."`) and `element.onclick = fn` are older patterns; they overwrite previous handlers.

## Common events

- Mouse: `click`, `mousedown`, `mouseup`, `mousemove`, `mouseover`, `mouseout`
- Form/input: `input`, `change`, `submit`
- Keyboard: `keydown`, `keyup`, `keypress` (deprecated)
- Focus: `focus`, `blur`



- Others: `scroll`, `resize`, `DOMContentLoaded`, `load`

## Event object

- Handler receives an `event` argument: `event.target` (origin element), `event.type` (event name), `event.preventDefault()` (stop the default action), `event.stopPropagation()` (prevent further bubbling), `event.stopImmediatePropagation()` (stop other listeners on same element).
- `event.currentTarget` is the element the listener is attached to (useful in delegation).

## Event phases

- Capturing phase:** root → target (rarely used)
- Target phase:** event hits target
- Bubbling phase:** event bubbles up from target to root (default for most events)
- You can register a listener to the capturing phase by passing `{ capture: true }` to `addEventListener`.

## Event delegation

- Instead of adding listeners to many child elements, attach one listener to a common ancestor and check `event.target` to act upon specific children. Great for dynamic lists.

```
js

document.querySelector('#todoList').addEventListener('click', (e) => {
  const btn = e.target.closest('.delete-btn');
  if (!btn) return;
  const item = btn.closest('.todo-item');
  item.remove();
});
```

## Confusions to avoid

- `event.target` vs `event.currentTarget`: `target` is where the event originated; `currentTarget` is where the listener is attached.
- Capturing vs bubbling: bubbling is common; capture is less used.

## Practice ideas

- Live character counter (listen `input` events)
- Delegated handler on todo list (adds/delete/edit items without adding listeners per item)

# 3. Forms and Form Validation

## Reading values

- Text inputs, textarea, select: use `.value`

```
js
```

```
const name = document.querySelector('#name').value;
```

- For checkboxes/radios: `.checked` or `.value` appropriately.

## Prevent default submission

- For custom handling, intercept submit:

```
js
```

```
form.addEventListener('submit', (e) => {  
  e.preventDefault();  
  // validate & send via fetch or show errors  
});
```

## Inline HTML validation vs JS validation

- HTML attributes: `required`, `minlength`, `maxlength`, `pattern` — helpful for quick native validation and accessibility.
- JS-based validation: more flexible, can present custom error messages and complex rules.
- Use both: native for quick constraints + JS for richer UX.

## Showing error messages conditionally

- On blur/submit show messages near fields; update aria attributes for accessibility: `aria-invalid="true"`, and `role="alert"` for message containers.

## Pattern attribute vs custom regex

- `pattern` attribute in HTML only handles simple regex and basic browser messages.
- For complex validations (password strength, cross-field checks), use JS and custom regex.

## Common confusions

- `value` vs `textContent`: input elements have `.value`. `textContent` is for general elements like `<div>` and `<p>`.
- Form submission vs button click: `<button type="submit">` inside a form triggers `submit` event. Clicking a plain `<button>` without type defaults to `submit` in some contexts — best to explicitly set `type="button"` when not submitting.
- Using `form.submit()` programmatically bypasses `submit` event listeners — call carefully.

## Sanitization

- Always sanitize and validate again on the server. Client-side validation is UX only.

## Practice

- Email/password validator: check format, length, show inline errors.
- Interactive feedback form: live validation as user types, error highlights, success summary.

## 4. Timers and Intervals

### APIs

- `setTimeout(fn, ms)` — run `fn` once after `ms` milliseconds. Returns a timer id you can pass to `clearTimeout(id)`.
- `setInterval(fn, ms)` — run `fn` every `ms` milliseconds until `clearInterval(id)`.

### Real uses

- Delays for UI actions (e.g., show a toast for 3s then hide).
- Auto-refreshing data (but prefer web sockets or careful polling).
- Animations (use `requestAnimationFrame` for per-frame animation instead of short intervals).

### setInterval vs recursive setTimeout

- `setInterval` tries to schedule periodically but if `fn` runtime exceeds interval, calls can pile up or drift.
- Recursive `setTimeout` runs `fn` then schedules the next call at the end — safer for tasks where you must wait for completion each time (e.g., fetch then wait).

```
js

function loop() {
  doWork().then(() => {
    setTimeout(loop, 1000);
  });
}

loop();
```

### Confusion

- Using `setInterval` when you need sequential operations can cause overlapping runs. Prefer `setTimeout` recursion for controlled sequencing.

### Practice

- Countdown timer (show mm:ss).
- Auto-hide an alert after 3s.

## 5. localStorage, sessionStorage, and Cookies

### APIs and behavior

- `localStorage` (persistent across browser restarts, per origin): `.setItem(key, value)`, `.getItem(key)`, `.removeItem(key)`, `.clear()`.

- `sessionStorage` (keeps data for the tab session only; cleared when tab closes) — same API as `localStorage`.
- Cookies: `document.cookie = "name=value; path=/; expires=Thu, 01 Jan 2099 00:00:00 GMT; Secure; SameSite=Lax"` — manual string format; limited size; sent to server on each request (so sensitive data should not be stored in cookies without flags and security).

## Strings vs JSON

- Storage APIs only store strings:

```
js

localStorage.setItem('theme', 'dark');
// For objects:
localStorage.setItem('user', JSON.stringify({ name: 'Dileep' }));
const user = JSON.parse(localStorage.getItem('user'));
```

- Always handle `null` from `getItem` and catch `JSON.parse` exceptions.

## Cookies details

- Cookies require manual encoding/decoding: `encodeURIComponent` / `decodeURIComponent` for values.
- To set expiration use `expires` or `max-age`.
- Cookies are included with HTTP requests (useful for session auth that the server reads).
- For client-only storage, prefer `localStorage` / `sessionStorage`.

## Security & use-cases

- Use `localStorage` for client-side preferences (theme, cached settings).
- Use cookies for server-read session tokens (HTTPOnly cookies are best for session auth — cannot be accessed via JavaScript).
- Avoid storing highly sensitive tokens in `localStorage` (XSS risk). Use secure, same-site cookies where possible.

## Confusions

- Why only strings? Simpler design; you must `JSON.stringify` objects.
- Cookies require manual expiration & encoding; `localStorage` has no built-in expiry.

## Practice

- Save theme preference in `localStorage` and apply on page load.
- Login form that “remembers” username using storage.

## Bonus Practice Projects — implementation focus & what to learn

Below are the project ideas from the slides with concrete implementation steps and learning goals.

## DOM Interactivity

### Todo App (add / delete / edit)

- Features: add item, edit item inline, delete item, mark complete, filter (all/active/completed).
- Data model: maintain array of todo objects in JS; persist to `localStorage`.
- Key skills: DOM creation, event delegation, `.value`, `contentEditable` or edit form, toggling classes, `localStorage` persistence, re-render vs incremental DOM updates.
- Bonus: keyboard shortcuts, drag-and-drop reorder.

### Accordion / Collapsible FAQ

- Single listener on container, toggle `.open` class on clicked header, animate height via CSS transitions (max-height trick).
- Accessibility: set ARIA attributes: `aria-expanded`, `aria-controls`.

## Event Handling

### Real-time search filter

- Input `input` event to filter list items by text.
- Debounce user input so heavy filtering doesn't run too often.

```
js
```

```
function debounce(fn, ms) { /*...*/ }
```

- Show “no results” message.

### Custom dropdown menu

- Click to open/close; keyboard navigation (up/down/enter/escape).
- Close on outside click (use document listener and `closest` to check).

## Form Management

### Survey form with multiple fields and validation

- Multi-step or single page; required fields, conditional fields, file uploads (validate size/type).
- Show progress, highlight invalid fields, focus first invalid field on submit.
- Use `FormData` if you're sending to server.

## Storage Integration

## Notes App (save text to localStorage)

- Add, edit, delete notes. Save timestamp. Search and sort.
- Use `localStorage` to persist notes. Keep an `id` for each note (timestamp or UUID).
- When loading, `JSON.parse` and handle errors.

## Dark mode toggle saved across sessions

- Toggle class on `document.documentElement`. Save preference to `localStorage`. On page load, read and apply before paint to avoid flicker.

## Helpful code snippets (copy-paste ready)

### Query & loop (static NodeList):

js

```
const buttons = document.querySelectorAll('.my-btn');
buttons.forEach(btn => btn.addEventListener('click', () => console.log('clicked')));
```

### Event delegation (todo list example):

js

```
const list = document.querySelector('#todo');
list.addEventListener('click', (e) => {
  const deleteBtn = e.target.closest('.delete');
  if (deleteBtn) {
    const item = deleteBtn.closest('.todo-item');
    item.remove();
    // update storage etc.
  }
});
```

### Form submit with validation:

js

```
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const email = form.querySelector('[name="email"]').value.trim();
  if (!/^\S+@\S+\.\S+$/i.test(email)) {
    showError('Invalid email');
    return;
  }
});
```

```
// proceed: fetch('/submit', { method: 'POST', body: new FormData(form) })
});
```

## Debounce helper:

```
js

function debounce(fn, wait=300) {
  let t;
  return function (...args) {
    clearTimeout(t);
    t = setTimeout(() => fn.apply(this, args), wait);
  };
}
```

## localStorage object safe getter:

```
js

function storageGet(key, fallback=null) {
  try {
    const raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : fallback;
  } catch (err) {
    console.error('storage parse error', err);
    return fallback;
  }
}

function storageSet(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
```

## Countdown timer:

```
js

function startCountdown(seconds, onTick, onEnd) {
  let remaining = seconds;
  const id = setInterval(() => {
    remaining--;
    onTick(remaining);
    if (remaining <= 0) {
      clearInterval(id);
    }
  }, 1000);
}
```

```
    onEnd();  
  }  
}, 1000);  
return () => clearInterval(id); // returns cancel fn  
}
```

## Practical tips & best practices

- **Prefer** `.classList` **over** `.style` for toggling UI classes.
- **Use event delegation** for lists and dynamic content to avoid many listeners and memory leaks.
- **Avoid XSS**: never `innerHTML` untrusted input. Use `textContent` or sanitize first.
- **Graceful degradation**: combine native HTML validation with JS enhancements for better accessibility and progressive enhancement.
- **Keep storage size in mind**: `localStorage` is limited by browser (few MBs). Use `indexedDB` for larger offline needs.
- **Test performance**: avoid frequent DOM reflows (reading layout properties like `offsetHeight` causes reflow).
- **Accessibility**: add keyboard support and ARIA attributes to interactive widgets.

If you want, I can:

- convert the above into a **printable cheat-sheet PDF**,
- produce a **step-by-step implementation** of one practice project (e.g., Todo app) with file structure, HTML/CSS/JS, and comments,
- or create short **live-coding exercises** and unit tests for each topic.

Which of those would you like next?